

Experiment 'p_6_4_sum_without_smallest_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_4_sum_without_smallest_seq

Natural language statement: null

Method signature: p_6_4_sum_without_smallest_seq(inputs: seq<int>) returns (sum: int)

Ensures

- `sum == sum_array(inputs) - find_smallest(inputs)`

Requires

- `|inputs| >= 1`

Functional Code Given

```
function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
}

function sum_array(inputs: seq<int>): int
{
```

```
if |inputs| == 0 then 0
else inputs[0] + sum_array(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_6_4_sum_without_smallest_seq(inputs: seq<int>) returns (sum: int)`

The method should respect the following contract:

`requires |inputs| >= 1, ensures sum == sum_array(inputs) - find_smallest(inputs)`

The contract uses the following dafny code:

```
function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
}

function sum_array(inputs: seq<int>): int
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOEbuaDOuCBgb6f6Z2DE0gDe7Zl3

Model: gpt-5.5-2026-04-23

Created at: 1780663601

Finish reason: stop

Usage: 690 tokens in, and 6803 tokens out

```
{
  var i := |inputs| - 1;
  assert 0 <= i < |inputs|;
  var smallest := inputs[i];
  sum := 0;

  assert i + 1 == |inputs|;
  assert inputs[i..] == [smallest];
  assert find_smallest(inputs[i..]) == smallest;
  assert sum_array(inputs[i..]) == smallest;

  while 0 < i
    invariant 0 <= i < |inputs|
    invariant smallest == find_smallest(inputs[i..])
    invariant sum == sum_array(inputs[i..]) - smallest
    decreases i
  {
    var oldI := i;
    var j := oldI - 1;
    assert 0 <= j < oldI < |inputs|;
    var x := inputs[j];
    var oldSmallest := smallest;
    var oldSum := sum;

    assert oldSmallest == find_smallest(inputs[oldI..]);
    assert oldSum == sum_array(inputs[oldI..]) - oldSmallest;
    assert oldI == j + 1;
    assert inputs[j..oldI] == [x];
    assert inputs[j..] == inputs[j..oldI] + inputs[oldI..];
    assert inputs[j..] == [x] + inputs[oldI..];
    assert (inputs[j..])[0] == x;
    assert (inputs[j..])[1..] == inputs[oldI..];
    assert 2 <= |inputs[j..]|;
    assert sum_array(inputs[j..]) == x + sum_array(inputs[oldI..]);

    if x <= oldSmallest {
      assert find_smallest(inputs[j..]) == x;
      sum := oldSum + oldSmallest;
      smallest := x;
      assert sum == sum_array(inputs[oldI..]);
      assert sum == sum_array(inputs[j..]) - smallest;
    } else {
      assert oldSmallest < x;
      assert find_smallest(inputs[j..]) == oldSmallest;
      sum := oldSum + x;
      smallest := oldSmallest;
      assert sum == sum_array(inputs[j..]) - smallest;
    }
  }
}
```

```

    i := j;
    assert smallest == find_smallest(inputs[i..]);
    assert sum == sum_array(inputs[i..]) - smallest;
  }

  assert i == 0;
  assert inputs[i..] == inputs;
  assert sum == sum_array(inputs) - find_smallest(inputs);
}

```

Final Program

```

function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
  }
}

function sum_array(inputs: seq<int>): int
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}

method p_6_4_sum_without_smallest_seq(inputs: seq<int>) returns (sum: int)
  requires |inputs| >= 1
  ensures sum == sum_array(inputs) - find_smallest(inputs)
{
  var i := |inputs| - 1;
  assert 0 <= i < |inputs|;
  var smallest := inputs[i];
  sum := 0;

  assert i + 1 == |inputs|;
  assert inputs[i..] == [smallest];
  assert find_smallest(inputs[i..]) == smallest;
  assert sum_array(inputs[i..]) == smallest;

  while 0 < i
    invariant 0 <= i < |inputs|
    invariant smallest == find_smallest(inputs[i..])

```

```

invariant sum == sum_array(inputs[i..]) - smallest
decreases i
{
  var oldI := i;
  var j := oldI - 1;
  assert 0 <= j < oldI < |inputs|;
  var x := inputs[j];
  var oldSmallest := smallest;
  var oldSum := sum;

  assert oldSmallest == find_smallest(inputs[oldI..]);
  assert oldSum == sum_array(inputs[oldI..]) - oldSmallest;
  assert oldI == j + 1;
  assert inputs[j..oldI] == [x];
  assert inputs[j..] == inputs[j..oldI] + inputs[oldI..];
  assert inputs[j..] == [x] + inputs[oldI..];
  assert (inputs[j..])[0] == x;
  assert (inputs[j..])[1..] == inputs[oldI..];
  assert 2 <= |inputs[j..]|;
  assert sum_array(inputs[j..]) == x + sum_array(inputs[oldI..]);

  if x <= oldSmallest {
    assert find_smallest(inputs[j..]) == x;
    sum := oldSum + oldSmallest;
    smallest := x;
    assert sum == sum_array(inputs[oldI..]);
    assert sum == sum_array(inputs[j..]) - smallest;
  } else {
    assert oldSmallest < x;
    assert find_smallest(inputs[j..]) == oldSmallest;
    sum := oldSum + x;
    smallest := oldSmallest;
    assert sum == sum_array(inputs[j..]) - smallest;
  }

  i := j;
  assert smallest == find_smallest(inputs[i..]);
  assert sum == sum_array(inputs[i..]) - smallest;
}

assert i == 0;
assert inputs[i..] == inputs;
assert sum == sum_array(inputs) - find_smallest(inputs);
}

```

Total Token Usage

Input tokens: 690

Output tokens: 6803

Reasoning tokens: 6142

Sum of ‘total tokens’: 7493

Experiment Timings

Overall Experiment started at 1780663599543, ended at 1780663764744, lasting 165201ms (165.20 seconds)

Iteration #1 started at 1780663599610, ended at 1780663764744, lasting 165134ms (165.13 seconds)